

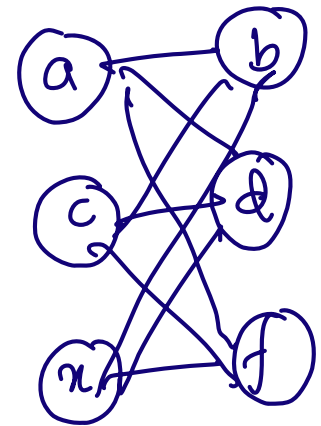
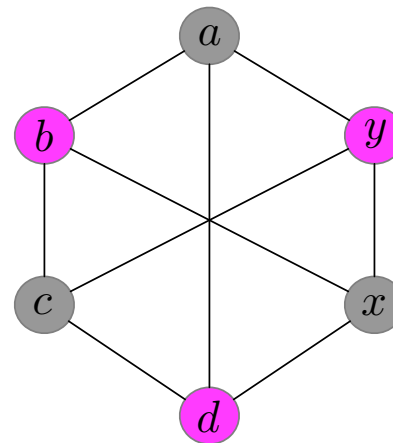
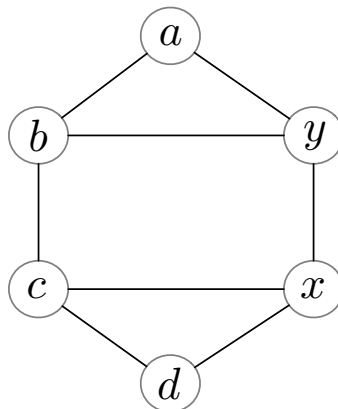
Application of BFS – Testing Bipartiteness Graph Algorithms

Diptapriyo Majumdar

March 16, 2023

Bipartite Graph:

- An undirected graph $G(V, E)$ is said to be bipartite if $V = A \uplus B$ such that for every edge $(u, v) \in E$, $u \in A$ if and only if $v \in B$.
- **Alternate characterization:** An undirected graph is bipartite if it has no cycle with odd number of vertices.
- In other words, a bipartite graph is a graph with no odd cycle.



BFS(G, s)

for all $u \in V(G)$ do

| Color u white;

end

EnQueue(Q, s);

Color u grey; $Layer(s) = 0$; Initialize queue $Q \leftarrow \emptyset$; $parent(s) = NULL$;

while Q is not empty do

| $u \leftarrow DeQueue(Q)$;

| for each $v \in Adj(u)$ do

| | if v is white then

| | | Color v grey; $Layer(v) = Layer(u) + 1$;

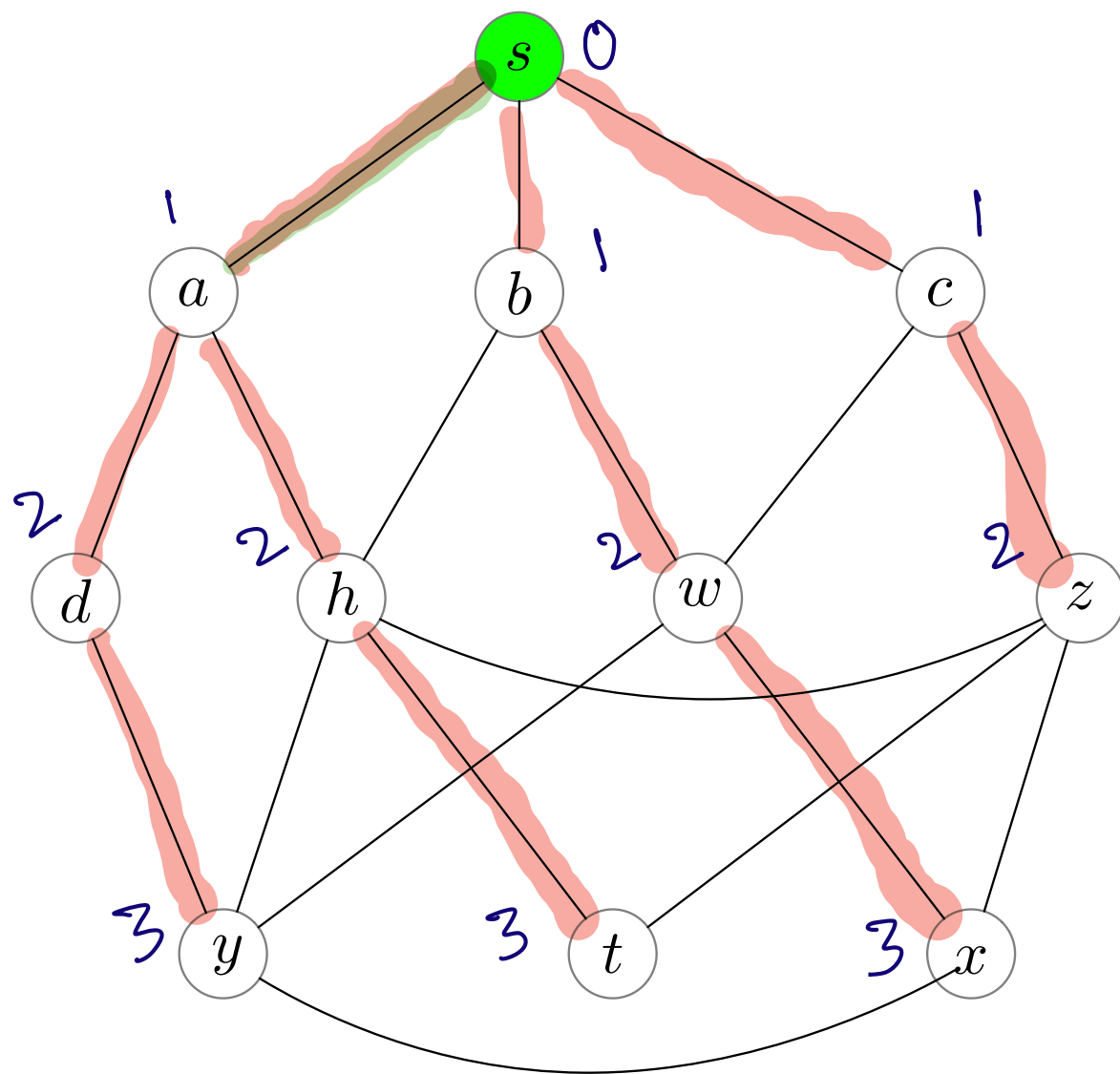
| | | $parent(v) = u$; $EnQueue(Q, v)$;

| | end

| end

| Color u black;

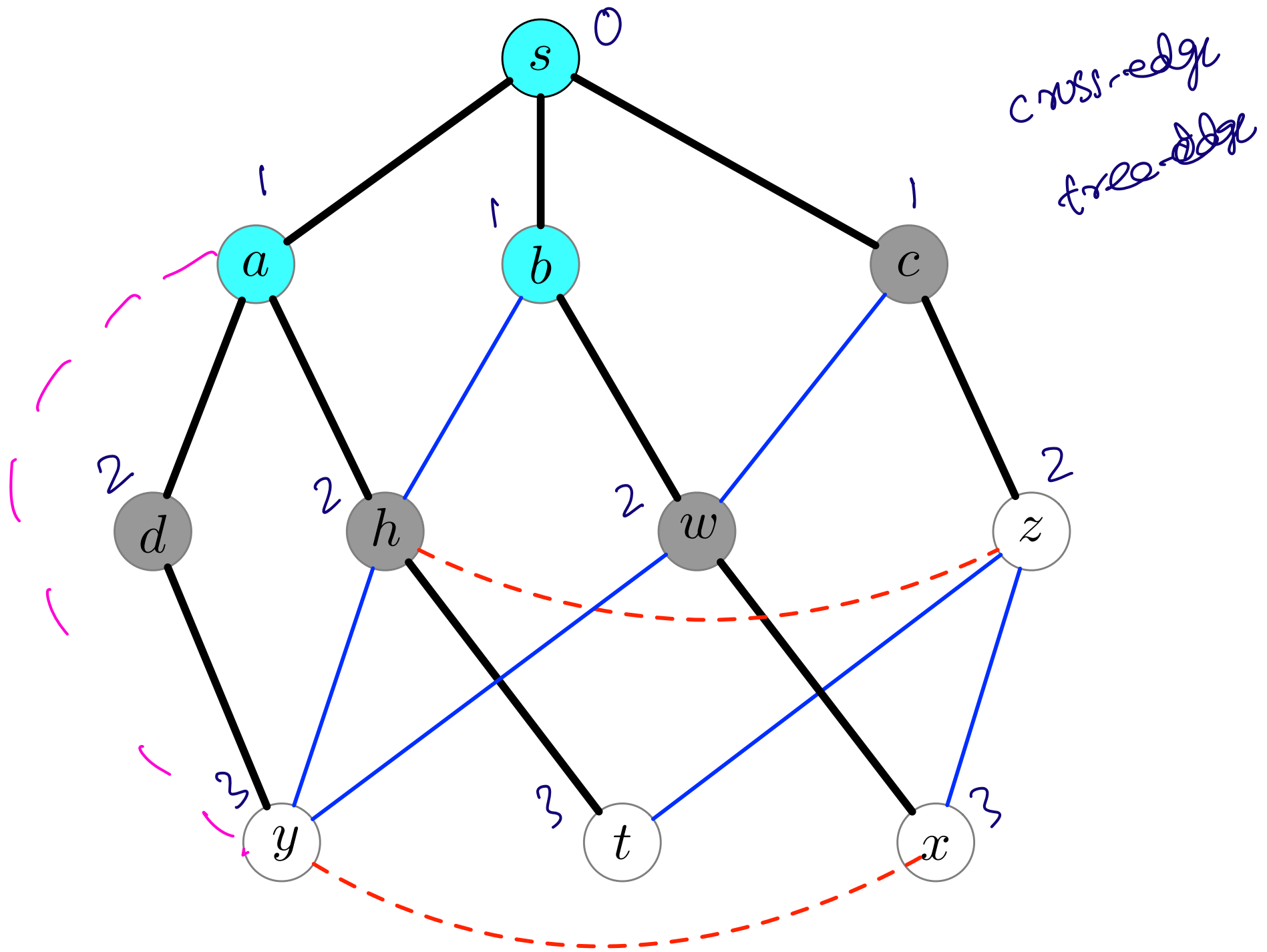
end

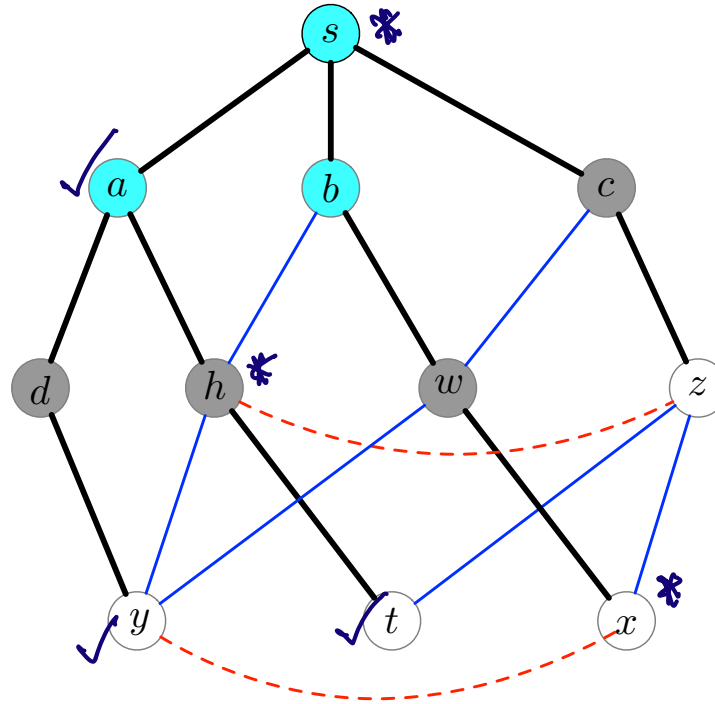


Some facts on BFS

Let G be a connected undirected graph.

- $\text{dist}(s, u)$ is the number of edges in a shortest path from s to u in G .
- For any edge $(u, v) \in E$, $\text{dist}(s, u) \leq \text{dist}(s, v) + 1$.
- BFS starting from s terminates with values $\text{Layer}(v) \geq \text{dist}(s, v)$ for all $v \in V$.
- If G is connected, then $\text{BFS}(G, s)$ terminates with $\text{dist}(s, v) = \text{Layer}(v)$ for all vertices $v \in V$.
- Every edge $(u, v) \in E$ satisfies $\text{Layer}(u) \leq \text{Layer}(v)$.

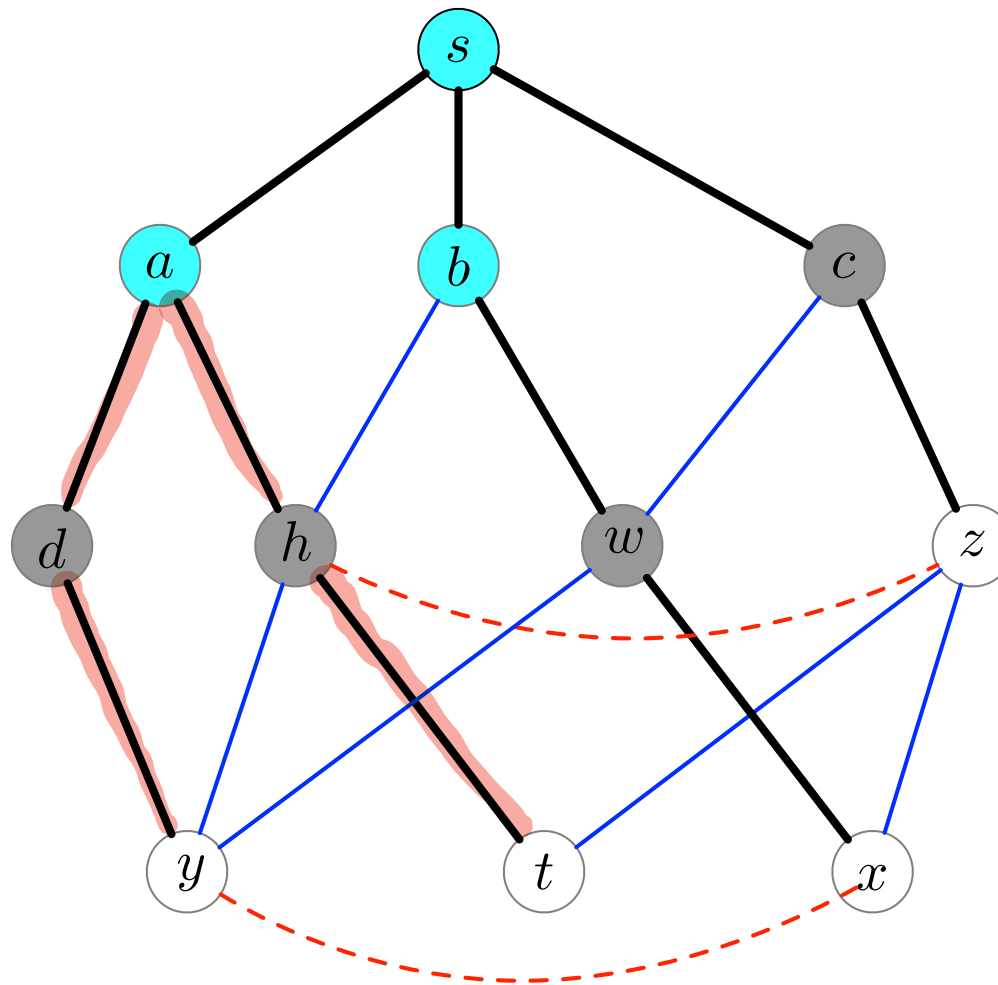




- BFS cannot have back-edge or forward edge.
- **Tree Edge:** An edge (u, v) is a tree-edge if $\text{dist}(s, v) = \text{dist}(s, u) + 1$. In other words, $\text{Layer}(v) = \text{Layer}(u) + 1$.
- **Cross Edge:** An edge (u, v) is a cross-edge if $\text{Layer}(v) = \text{Layer}(u)$ or $\text{Layer}(v) = \text{Layer}(u) + 1$. rooted
- **Lowest Common Ancestor (LCA):** Given a tree T , the lowest common ancestor of u and v in T is denoted as $\text{LCA}_T(u, v)$.

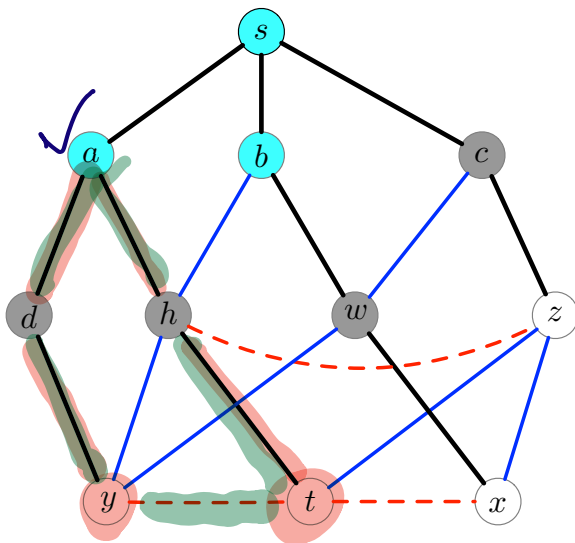
Characterization of Bipartiteness

- An undirected graph G is bipartite if and only if $BFS(G, s)$ does not have any (cross)-edge (u, v) satisfies $Layer(u) = Layer(v)$.

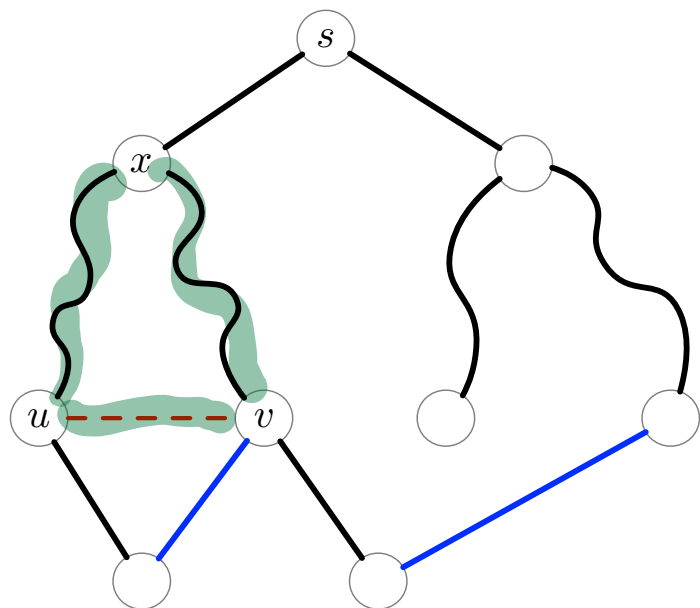


- **Necessary and Sufficient Condition:** An undirected graph G is bipartite if and only if $BFS(G, s)$ does not have any (cross)-edge (u, v) satisfies $Layer(u) = Layer(v)$.

Proof (Sketch):



- $(\Rightarrow)$ Let (y, t) be a cross-edge with $Layer(y) = Layer(t)$.
- Let a be the lowest common ancestor (LCA) of y and t in the BFS-Tree T .
- Then, there is a path $P_{a,y}$ from a to y containing $dist(s, y) - dist(s, a)$ edges.
- There is a path $P_{a,t}$ from a to t containing $dist(s, t) - dist(s, a)$ edges.
- $P_{a,t} \cap P_{a,y}$ are pairwise vertex-disjoint. (except a)
- Consider the number of edges in $E(P_{a,t}) \cup E(P_{a,y}) \cup \{(y, t)\}$.

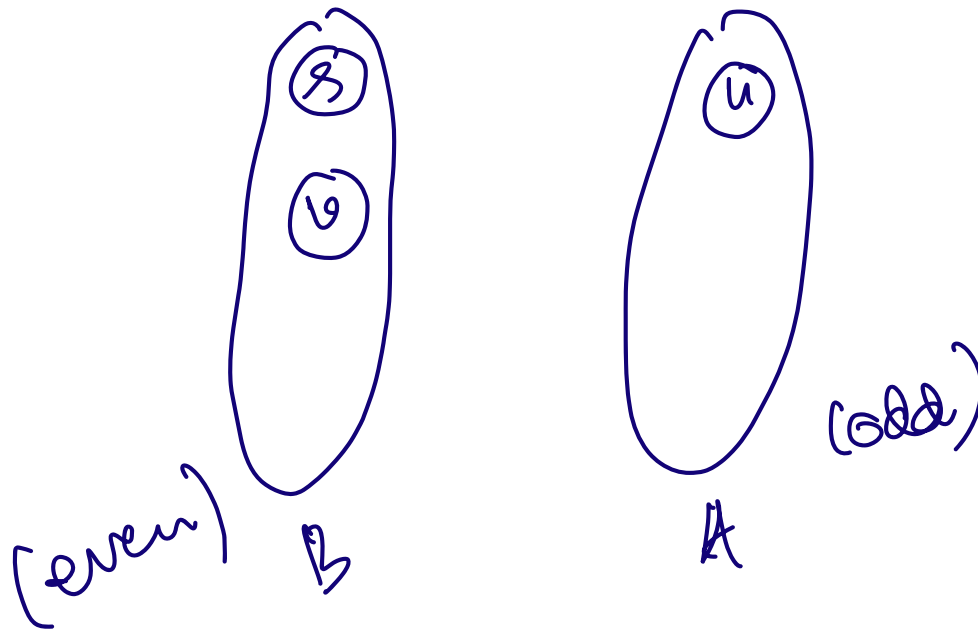


- Let x be the **lowest common ancestor (LCA)** of u and v in the BFS-Tree T , i.e.
 $x = LCA_T(u, v)$.
- Then, there is a path $P_{x,u}$ from x to u containing $Layer(u) - Layer(x)$ edges.
- There is a path $P_{x,v}$ from x to v containing $Layer(v) - Layer(x)$ edges.
- $P_{x,u} \cap P_{x,v}$ are pairwise vertex-disjoint.
- The number of edges in $E(P_{x,u}) \cup E(P_{x,v}) \cup \{(u, v)\}$ is $[Layer(u) + Layer(v) - 2Layer(x)] + 1$.
- $P_{x,u} \cup P_{x,v} \cup \{(u, v)\}$ is a cycle with odd number of vertices.

- **Necessary and Sufficient Condition:** An undirected graph G is bipartite if and only if $BFS(G, s)$ does not have any (cross)-edge (u, v) satisfying $Layer(u) = Layer(v)$.

Proof (continued):

- $(\Leftarrow)$ Suppose that $BFS(G, s)$ has no cross-edge (u, v) satisfying $Layer(u) = Layer(v)$.
- Then, construct a bipartition $V(G) = A \uplus B$ as follows.



- **Necessary and Sufficient Condition:** An undirected graph G is bipartite if and only if $BFS(G, s)$ does not have any (cross)-edge (u, v) satisfies $Layer(u) = Layer(v)$.

Proof (continued):

- $(\Leftarrow)$ Suppose that $BFS(G, s)$ has no cross-edge (u, v) satisfying $Layer(u) = Layer(v)$.
- Then, construct a bipartition $V(G) = A \uplus B$ as follows.
- Invoke $BFS(G, s)$ and compute $Layer(v)$ for all $v \in V$.
- Initialize $A, B \leftarrow \emptyset$. If $Layer(v)$ is odd, then $A \leftarrow A \cup \{v\}$. If $Layer(v)$ is even, then $B \leftarrow B \cup \{v\}$.
- There is no edge (u, v) such that $Layer(u) = Layer(v)$.
- Every edge (u, v) satisfies $Layer(u) = Layer(v) + 1$.
- There is no edge (u, v) such that $Layer(u) \geq Layer(v) + 2$.
- Hence, any edge (u, v) of G satisfies $u \in A$ if and only if $v \in B$.

BFS(G, s)

For all $v \in V$, color v white;

Color u grey; $Layer(s) = 0$; Initialize queue $Q \leftarrow \emptyset$; $parent(s) = NULL$;

while Q is not empty **do**

$u \leftarrow DeQueue(Q)$;

for each $v \in Adj(u)$ **do**

if v is white **then**

 Color v grey; $Layer(v) = Layer(u) + 1$;

$parent(v) = u$; $EnQueue(Q, v)$;

end

else

if $Layer(u) = Layer(v)$ **then**

 Output *there is an odd cycle*;

end

end

end

 Color u black;

end

Output G is bipartite graph.

Algorithm 2

- **Stage 1:** Invoke $BFS(G, s)$ and obtain compute $Layer(v)$ for all $v \in V$.
- **Stage 2:** For every edge $(u, v) \in E$, check if $Layer(u) = Layer(v)$.
- If for all $(u, v) \in E$, it holds that $Layer(u) \neq Layer(v)$, then output that G is bipartite graph.
- Otherwise, there is $(u, v) \in E$ such that $Layer(u) = Layer(v)$. Output that G is not bipartite.

- Section 3.4 of Algorithm Design (Kleinberg and Tardos).